

Bài 3: Câu lệnh rẽ nhánh

Fundamental Programming Concepts in Python

Faculty of Data Science and Artificial Intelligence

National Economics University

DSAI1003

Dựa trên Chương 3 – *Decisions, Python for Everyone.*

Vì sao cần câu lệnh rẽ nhánh?

Ở Bài 2, chương trình chủ yếu chạy theo thứ tự từ trên xuống dưới.

Trong thực tế, chương trình cần **đưa ra quyết định**: chọn hành động khác nhau tùy theo dữ liệu.

Ví dụ

- áp dụng giảm giá;
- kiểm tra dữ liệu nhập;
- phân loại điểm;
- chọn công thức tính thuế;
- xử lý chuỗi theo điều kiện.

Đánh giá điều kiện $\rightarrow$ Đúng/Sai $\rightarrow$ Chọn nhánh

Luồng điều khiển

Điều kiện đúng $\Rightarrow$ thực hiện hành động A

Điều kiện sai $\Rightarrow$ thực hiện hành động B

Sau khi một nhánh được thực hiện, chương trình tiếp tục với các lệnh phía sau cấu trúc rẽ nhánh.

Mục tiêu học tập

Sau bài này, sinh viên có thể:

- sử dụng `if`, `if/else`, `if/elif/else`;
- dùng toán tử quan hệ để viết điều kiện;
- tổ chức khối lệnh bằng thụt lề đúng;
- thiết kế rẽ nhánh lồng nhau và nhiều phương án;
- sử dụng `and`, `or`, `not`;
- thiết kế ca kiểm thử cho các nhánh và giá trị biên;
- phân tích chuỗi và kiểm tra dữ liệu đầu vào.

Cấu trúc bài học

- 1 Câu lệnh `if`
- 2 Toán tử quan hệ
- 3 Rẽ nhánh lồng nhau
- 4 Nhiều phương án
- 5 Lưu đồ
- 6 Ca kiểm thử
- 7 Boolean và toán tử Boolean
- 8 Phân tích chuỗi
- 9 Kiểm tra dữ liệu đầu vào
- 10 Tổng kết và bài tập

Quyết định hai nhánh

Ví dụ: tòa nhà không dùng số tầng 13. Nếu người dùng chọn tầng hiển thị lớn hơn 13, tầng thực tế giảm đi 1.

```
if floor > 13 :  
    actualFloor = floor - 1  
else :  
    actualFloor = floor
```

Điểm cần nhớ

Chỉ **một nhánh** được thực hiện. Điều kiện đúng thì chạy nhánh `if`; điều kiện sai thì chạy nhánh `else`.

Cú pháp tổng quát

Có nhánh else

```
if condition :  
    statements1  
else :  
    statements2
```

Không có nhánh else

```
if condition :  
    statements
```

Nhánh else là tùy chọn. Dùng khi cần xử lý riêng trường hợp điều kiện sai.

Câu lệnh ghép và khối lệnh

```
if totalSales > 100.0 :  
    discount = totalSales * 0.05  
    totalSales = totalSales - discount  
    print("You received a discount of", discount)
```

- Dòng tiêu đề kết thúc bằng dấu hai chấm :.
- Các lệnh trong cùng một khối phải thụt lề cùng mức.
- Khối kết thúc khi mức thụt lề quay lại mức bên ngoài.

Quan trọng

Trong Python, thụt lề là một phần của cú pháp, không chỉ để trình bày đẹp hơn.

Ví dụ: mô phỏng thang máy

```
##  
# This program simulates an elevator panel that skips floor 13.  
#  
  
floor = int(input("Floor: "))  
  
if floor > 13 :  
    actualFloor = floor - 1  
else :  
    actualFloor = floor  
  
print("The elevator will travel to the actual floor", actualFloor)
```

Ví dụ chạy

```
Floor: 20  
The elevator will travel to the actual floor 19
```

Lỗi thường gặp: tab và thụt lề

Python yêu cầu các lệnh trong cùng khối có mức thụt lề nhất quán.

```
if score >= 50 :  
    print("Pass")  
    print("Continue to the next course")
```

Tránh lỗi

Không trộn tab và khoảng trắng. Nên cấu hình editor để phím Tab tạo 4 khoảng trắng.

Mẹo: tránh lặp mã trong các nhánh

Kém hơn

```
if floor > 13 :  
    actualFloor = floor - 1  
    print("Actual floor:", actualFloor)  
else :  
    actualFloor = floor  
    print("Actual floor:", actualFloor)
```

Tốt hơn

```
if floor > 13 :  
    actualFloor = floor - 1  
else :  
    actualFloor = floor  
  
print("Actual floor:", actualFloor)
```

Chủ đề bổ sung: biểu thức điều kiện

Python có dạng rút gọn cho một số quyết định đơn giản.

```
actualFloor = floor - 1 if floor > 13 else floor
```

Khi nào dùng?

- Khi hai nhánh đều là biểu thức ngắn.
- Không nên dùng nếu làm mã khó đọc.

Ví dụ bổ sung: miễn phí giao hàng

Một cửa hàng miễn phí giao hàng nếu đơn hàng từ 500,000 đồng trở lên.

```
orderValue = float(input("Order value: "))

if orderValue >= 500000 :
    shippingFee = 0
else :
    shippingFee = 30000

total = orderValue + shippingFee
print("Shipping fee:", shippingFee)
print("Total:", total)
```

Điểm học được

Điều kiện `orderValue >= 500000` chia bài toán thành hai trường hợp rõ ràng: được miễn phí và không được miễn phí.

Quiz 3.1: if/else

Chọn đáp án đúng.

- ❶ Với $x = 10$, điều kiện $x > 10$ có giá trị gì?
- ❷ Dấu `:` trong dòng `if condition` : dùng để làm gì?
- ❸ Nếu điều kiện của `if` sai và không có `else`, chương trình làm gì?

Thảo luận nhanh

Hãy dự đoán trước khi chạy code. Đây là thói quen quan trọng khi học lập trình.

Đáp án Quiz 3.1

- ❶ False, vì 10 không lớn hơn 10.
- ❷ Dấu : báo hiệu bắt đầu một khối lệnh.
- ❸ Chương trình bỏ qua khối if và tiếp tục với lệnh phía sau.

Các toán tử quan hệ

Toán tử	Ý nghĩa
<	nhỏ hơn
<=	nhỏ hơn hoặc bằng
>	lớn hơn
>=	lớn hơn hoặc bằng
==	bằng nhau
!=	khác nhau

Kết quả của phép so sánh là một giá trị Boolean: True hoặc False.

Phép gán và phép so sánh bằng

Phép gán

```
floor = 13
```

Lưu giá trị 13 vào biến floor.

Phép kiểm tra bằng nhau

```
if floor == 13 :  
    print("No such floor")
```

Kiểm tra floor có bằng 13 hay không.

Lưu ý

Trong điều kiện, thường dùng ==, không dùng =.

So sánh số thực: không nên kiểm tra bằng tuyệt đối

Số thực trong máy tính có thể chứa sai số làm tròn rất nhỏ.

```
from math import sqrt

r = sqrt(2.0)
print(r * r)
```

Kết quả có thể là một giá trị rất gần 2, nhưng không đúng bằng 2.

Cách kiểm tra gần bằng

```
EPSILON = 1E-14
if abs(r * r - 2.0) < EPSILON :
    print("Close enough")
```

So sánh chuỗi

Python có thể so sánh chuỗi theo thứ tự từ điển.

```
if name < "M" :  
    print("First half of the directory")  
else :  
    print("Second half of the directory")
```

- So sánh diễn ra từng ký tự từ trái sang phải.
- Chữ hoa và chữ thường có mã ký tự khác nhau.
- Nên chuẩn hóa chuỗi bằng `upper()` hoặc `lower()` khi cần.

Cách làm: cài đặt một câu lệnh `if`

- 1 Xác định điều kiện rẽ nhánh.
- 2 Mô tả điều xảy ra khi điều kiện đúng.
- 3 Mô tả điều xảy ra khi điều kiện sai.
- 4 Kiểm tra toán tử quan hệ tại giá trị biên.
- 5 Loại bỏ phần xử lý lặp lại.
- 6 Kiểm thử cả hai nhánh.
- 7 Ghép thành chương trình Python.

Ví dụ minh họa: lấy phần giữa của chuỗi

Bài toán: nhập một chuỗi và lấy phần giữa. Nếu chuỗi có độ dài lẻ, lấy 1 ký tự giữa. Nếu độ dài chẵn, lấy 2 ký tự giữa.

```
string = input("Enter a string: ")
length = len(string)

if length % 2 == 1 :
    position = length // 2
    result = string[position]
else :
    position = length // 2
    result = string[position - 1] + string[position]

print("Middle:", result)
```

Ví dụ bổ sung: kiểm tra tuổi xem phim

Một rạp phim yêu cầu người xem từ 18 tuổi trở lên đối với phim nhãn 18+.

```
age = int(input("Age: "))  
  
if age >= 18 :  
    print("Ticket allowed")  
else :  
    print("Not allowed for this movie")
```

Giá trị biên

Với điều kiện `age >= 18`, cần kiểm tra ít nhất các giá trị 17, 18, 19.

Quiz 3.2: toán tử quan hệ

Dự đoán kết quả True/False.

Biểu thức	Kết quả?
<code>7 == 7.0</code>	
<code>7 != 7</code>	
<code>10 <= 10</code>	
<code>"An" < "Bình"</code>	
<code>3 + 4 > 10</code>	

Đáp án Quiz 3.2

Biểu thức	Kết quả
<code>7 == 7.0</code>	True
<code>7 != 7</code>	False
<code>10 <= 10</code>	True
<code>"An" < "Bình"</code>	True
<code>3 + 4 > 10</code>	False

Khi nào cần rẽ nhánh lồng nhau?

Một số bài toán cần quyết định theo nhiều tầng.

Ví dụ: tính thuế phụ thuộc vào:

- tình trạng hôn nhân;
- mức thu nhập;
- ngưỡng thuế tương ứng với từng nhóm.

Khi đó, nhánh bên ngoài chọn nhóm chính, nhánh bên trong xử lý điều kiện chi tiết hơn.

Ví dụ rẽ nhánh lồng nhau

```
if maritalStatus == "s" :  
    if income <= 32000 :  
        tax = 0.10 * income  
    else :  
        tax = 3200 + 0.25 * (income - 32000)  
else :  
    if income <= 64000 :  
        tax = 0.10 * income  
    else :  
        tax = 6400 + 0.25 * (income - 64000)
```

Cách đọc

Trước hết kiểm tra nhóm chính; sau đó, trong từng nhóm, kiểm tra ngưỡng thu nhập.

Mẹo: mô phỏng bằng tay

Hand-tracing

Mô phỏng chương trình trên giấy: ghi giá trị biến sau từng câu lệnh và đánh dấu nhánh được thực hiện.

Khi nào rất hữu ích?

- rẽ nhánh lồng nhau;
- biểu thức Boolean phức tạp;
- lỗi kết quả không rõ nguyên nhân;
- kiểm thử giá trị biên.

Ví dụ bổ sung: đăng nhập đơn giản

Một chương trình kiểm tra tên người dùng trước, sau đó mới kiểm tra mật khẩu.

```
userName = input("Username: ")
password = input("Password: ")

if userName == "admin" :
    if password == "12345" :
        print("Login successful")
    else :
        print("Wrong password")
else :
    print("Unknown username")
```

Cách đọc

Nhánh trong chỉ được xét khi nhánh ngoài đã đúng.

Quiz 3.3: truy vết rẽ nhánh lồng nhau

Với đoạn code sau, chương trình in gì nếu $x = 5, y = 2$?

```
if x > 0 :  
    if y > 0 :  
        print("A")  
    else :  
        print("B")  
else :  
    print("C")
```

Câu hỏi thêm

Kết quả thay đổi thế nào nếu $x = 5, y = -2$?

Đáp án Quiz 3.3

- Nếu $x = 5, y = 2$: in "A".
- Nếu $x = 5, y = -2$: in "B".
- Nếu $x \leq 0$: nhánh trong không được xét, chương trình in "C".

Câu lệnh `if/elif/else`

Khi có nhiều phương án loại trừ lẫn nhau, dùng `elif` để tránh nhiều `if` độc lập.

```
if condition1 :  
    statements1  
elif condition2 :  
    statements2  
elif condition3 :  
    statements3  
else :  
    statements4
```

Nguyên tắc

Python kiểm tra từ trên xuống. Nhánh đầu tiên có điều kiện đúng sẽ được thực hiện.

Ví dụ: phân loại động đất

```
if richter >= 8.0 :  
    description = "Most structures fall"  
elif richter >= 7.0 :  
    description = "Many buildings destroyed"  
elif richter >= 6.0 :  
    description = "Many buildings damaged"  
elif richter >= 4.5 :  
    description = "Damage to poorly constructed buildings"  
else :  
    description = "No destruction of buildings"
```

Điểm mẫu chốt

Các ngưỡng phải được đặt theo thứ tự phù hợp. Với ví dụ này, kiểm tra từ lớn xuống nhỏ.

Thứ tự điều kiện rất quan trọng

Sai ý định

```
if richter >= 4.5 :  
    description = "Damage"  
elif richter >= 6.0 :  
    description = "Many damaged"  
elif richter >= 8.0 :  
    description = "Most fall"
```

Vì sao sai?

Nếu richter = 8.5, điều kiện đầu tiên richter >= 4.5 đã đúng, nên các nhánh sau không bao giờ được xét.

elif khác gì nhiều if độc lập?

Loại trừ lẫn nhau

```
if score >= 90 :  
    grade = "A"  
elif score >= 80 :  
    grade = "B"  
elif score >= 70 :  
    grade = "C"
```

Nhiều kiểm tra độc lập

```
if hasLicense :  
    print("License OK")  
if hasInsurance :  
    print("Insurance OK")  
if hasRegistration :  
    print("Registration OK")
```

Ví dụ bổ sung: tính phí vận chuyển theo vùng

```
zone = input("Zone A/B/C: ").upper()

if zone == "A" :
    fee = 15000
elif zone == "B" :
    fee = 25000
elif zone == "C" :
    fee = 40000
else :
    fee = -1
    print("Invalid zone")

if fee >= 0 :
    print("Shipping fee:", fee)
```

Điểm học được

if/elif/else phù hợp khi các phương án loại trừ lẫn nhau.

Quiz 3.4: chọn cấu trúc phù hợp

Nên dùng `elif` hay nhiều `if` độc lập?

- 1 Phân loại điểm A/B/C/D/F.
- 2 Kiểm tra một đơn hàng có: mã giảm giá, freeship, quà tặng.
- 3 Phân loại mức độ động đất theo ngưỡng Richter.
- 4 Kiểm tra username: đủ dài, không có khoảng trắng, chỉ chữ/số.

Đáp án Quiz 3.4

- ❶ Dùng `if/elif/else`: các mức điểm loại trừ nhau.
- ❷ Dùng nhiều `if`: các ưu đãi có thể cùng xảy ra.
- ❸ Dùng `if/elif/else`: mỗi mức Richter thuộc một nhóm.
- ❹ Thường dùng nhiều điều kiện kết hợp hoặc nhiều `if`, vì cần kiểm tra nhiều lỗi khác nhau.

Lưu đồ: công cụ mô tả quyết định

Lưu đồ giúp nhìn rõ luồng điều khiển trước khi viết mã.

Các thành phần thường dùng

- điểm bắt đầu/kết thúc;
- bước xử lý;
- điều kiện rẽ nhánh;
- mũi tên thể hiện luồng đi.

Trong bài học này, lưu đồ chủ yếu dùng để mô tả quyết định hai nhánh và nhiều phương án.

Tránh luồng điều khiển kiểu “spaghetti”

Một chương trình tốt nên có luồng điều khiển dễ theo dõi.

Dấu hiệu cần tránh

- quá nhiều nhánh lồng nhau không cần thiết;
- điều kiện trùng lặp hoặc chồng lấn khó hiểu;
- cùng một xử lý bị copy ở nhiều nhánh;
- thứ tự kiểm tra không rõ lý do.

Ca kiểm thử: kiểm tra từng nhánh

Bao phủ nhánh

Mỗi nhánh quan trọng của chương trình cần có ít nhất một ca kiểm thử.

Ví dụ với điều kiện `floor > 13`

- `floor = 20`: nhánh đúng.
- `floor = 10`: nhánh sai.
- `floor = 13`: giá trị biên.
- `floor = 14`: ngay sau biên.

Giá trị biên

Lỗi thường xuất hiện gần các ngưỡng so sánh.

Nếu điều kiện là `score >= 50`, hãy kiểm tra:

Giá trị	Ý nghĩa
49	ngay dưới ngưỡng
50	đúng tại ngưỡng
51	ngay trên ngưỡng

Thiết kế ca kiểm thử trước khi lập trình giúp phát hiện lỗi logic sớm hơn.

Ví dụ bổ sung: bảng ca kiểm thử cho phân loại điểm

Điểm	Trường hợp	Kết quả mong đợi
- 1	ngoài miền dưới	Invalid
0	biên dưới	F
59	ngay dưới 60	F
60	biên D	D
89	ngay dưới A	B
90	biên A	A
101	ngoài miền trên	Invalid

Nhận xét

Bảng kiểm thử nên bao gồm cả nhánh hợp lệ, nhánh không hợp lệ và các giá trị sát ngưỡng.

Hoạt động nhanh: thiết kế ca kiểm thử

Bài toán: vé trẻ em nếu tuổi dưới 12; vé người lớn nếu từ 12 đến 59; vé người cao tuổi nếu từ 60 trở lên. Tuổi âm là không hợp lệ.

Yêu cầu

Hãy đề xuất ít nhất 6 ca kiểm thử, gồm:

- một ca không hợp lệ;
- các ca ngay dưới, đúng tại, và ngay trên mỗi ngưỡng;
- một ca đại diện cho từng nhóm vé.

Biến Boolean

Một biến Boolean lưu một trong hai giá trị: True hoặc False.

```
isValid = score >= 0 and score <= 100

if isValid :
    print("Valid score")
else :
    print("Invalid score")
```

Lợi ích

Biến Boolean giúp đặt tên cho một điều kiện, làm chương trình dễ đọc hơn.

Toán tử and

and đúng khi **cả hai** điều kiện đều đúng.

A	B	A and B
False	False	False
False	True	False
True	False	False
True	True	True

Ví dụ

```
score >= 0 and score <= 100
```

Toán tử or

or đúng khi **ít nhất một** điều kiện đúng.

A	B	A or B
False	False	False
False	True	True
True	False	True
True	True	True

Ví dụ

```
ch == "Y" or ch == "y"
```

Toán tử not và thứ tự ưu tiên

not đảo giá trị Boolean.

```
if not isValid :  
    print("Invalid input")
```

Thứ tự ưu tiên thường gặp:

Ưu tiên	Toán tử
cao	toán tử quan hệ như <, == not and
thấp	or

Dùng ngoặc để làm rõ ý định.

Lỗi thường gặp: nhầm and với or

Muốn kiểm tra điểm nằm ngoài khoảng 0 đến 100:

Sai

```
if score < 0 and score > 100 :  
    print("Invalid")
```

Không có số nào vừa nhỏ hơn 0 vừa lớn hơn 100.

Đúng

```
if score < 0 or score > 100 :  
    print("Invalid")
```

Một trong hai điều kiện xảy ra là đủ.

Đánh giá ngắn mạch

Python đánh giá `and` và `or` từ trái sang phải và dừng khi đã biết kết quả.

```
if quantity > 0 and price / quantity < 10 :  
    print("Good price")
```

Nếu `quantity > 0` sai, biểu thức thứ hai không được đánh giá. Điều này tránh lỗi chia cho 0.

Ý nghĩa

Thứ tự các điều kiện có thể ảnh hưởng đến cả tính đúng đắn và độ an toàn của chương trình.

Định luật De Morgan

De Morgan giúp biến đổi biểu thức phủ định.

Biểu thức		Tương đương
<code>not (A and B)</code>	$\Leftrightarrow$	<code>not A or not B</code>
<code>not (A or B)</code>	$\Leftrightarrow$	<code>not A and not B</code>

Ví dụ

`not (score >= 0 and score <= 100)` tương đương với `score < 0 or score > 100`.

Quiz 3.5: Boolean

Dự đoán kết quả nếu `score = 75` và `absent = 2`.

- ❶ `score >= 50 and absent <= 3`
- ❷ `score >= 80 or absent <= 3`
- ❸ `not (score < 50)`
- ❹ `score < 0 or score > 100`

Câu hỏi thêm

Điều kiện nào phù hợp để kiểm tra `score` nằm trong đoạn 0–100?

Đáp án Quiz 3.5

Với `score = 75`, `absent = 2`:

- ① True
- ② True
- ③ True
- ④ False

Điều kiện kiểm tra điểm hợp lệ:

```
score >= 0 and score <= 100
```

Ví dụ bổ sung: điều kiện nhận học bổng

Sinh viên được xét học bổng nếu GPA từ 3.2 trở lên và số tín chỉ đạt ít nhất 15.

```
gpa = float(input("GPA: "))
credits = int(input("Credits: "))

if gpa >= 3.2 and credits >= 15 :
    print("Scholarship candidate")
else :
    print("Not eligible")
```

Điểm học được

and dùng khi tất cả điều kiện con đều phải đúng.

Toán tử thành viên `in` và `not in`

```
if "@" in email :  
    print("May be an email address")  
  
if " " not in userName :  
    print("No spaces")
```

Ý nghĩa

`in` kiểm tra một chuỗi con có xuất hiện trong chuỗi lớn hơn hay không.

Một số phương thức chuỗi hữu ích

Phương thức	Công dụng
<code>s.startswith(t)</code>	kiểm tra bắt đầu bằng t
<code>s.endswith(t)</code>	kiểm tra kết thúc bằng t
<code>s.find(t)</code>	vị trí xuất hiện đầu tiên của t
<code>s.count(t)</code>	số lần xuất hiện của t
<code>s.lower()</code>	chuyển về chữ thường
<code>s.upper()</code>	chuyển về chữ hoa

Kiểm tra đặc điểm của chuỗi

```
if text.isdigit() :  
    number = int(text)  
  
if name.isalpha() :  
    print("Letters only")  
  
if code.isalnum() :  
    print("Letters and digits only")
```

- `isdigit()`: toàn chữ số.
- `isalpha()`: toàn chữ cái.
- `isalnum()`: chữ cái hoặc chữ số.

Ví dụ: khảo sát chuỗi con

```
string = input("Enter a string: ")
substring = input("Enter a substring: ")

position = string.find(substring)

if position == -1 :
    print("Not found")
else :
    print("First occurrence starts at", position)
    print("Number of occurrences:", string.count(substring))
```

Kết nối với rẽ nhánh

Phương thức chuỗi thường tạo ra điều kiện để chương trình quyết định bước xử lý tiếp theo.

Ví dụ bổ sung: kiểm tra mã sinh viên

Giả sử mã sinh viên hợp lệ nếu bắt đầu bằng "SV" và phần sau là chữ số.

```
studentID = input("Student ID: ").upper()

if studentID.startswith("SV") and studentID[2:].isdigit() :
    print("Valid student ID")
else :
    print("Invalid student ID")
```

Lưu ý

Cần cẩn thận nếu chuỗi quá ngắn. Khi dạy sâu hơn, nên bổ sung kiểm tra `len(studentID) >= 3`.

Quiz 3.6: phân tích chuỗi

Với `s = "Python 3"`, dự đoán kết quả:

- ❶ `"Py" in s`
- ❷ `s.startswith("py")`
- ❸ `s.upper().startswith("PY")`
- ❹ `s.isalpha()`
- ❺ `" " in s`

Đáp án Quiz 3.6

Với `s = "Python 3"`:

- ① True
- ② False, vì phân biệt hoa/thường.
- ③ True, vì đã chuẩn hóa bằng `upper()`.
- ④ False, vì có khoảng trắng và chữ số.
- ⑤ True.

Vì sao cần kiểm tra dữ liệu đầu vào?

Người dùng có thể nhập dữ liệu ngoài phạm vi mong đợi.

Nếu chương trình xử lý ngay mà không kiểm tra, có thể xảy ra:

- kết quả sai;
- lỗi chạy chương trình;
- thông báo khó hiểu;
- dữ liệu không nhất quán.

Nguyên tắc

Kiểm tra tính hợp lệ trước khi xử lý chính.

Kiểm tra miền giá trị

```
score = int(input("Score: "))

if score < 0 or score > 100 :
    print("Invalid score")
else :
    if score >= 50 :
        print("Pass")
    else :
        print("Fail")
```

Mẫu xử lý

Trước hết loại bỏ dữ liệu không hợp lệ; sau đó xử lý các trường hợp hợp lệ.

Kiểm tra lựa chọn nhập vào

```
choice = input("Continue? (Y/N): ")
choice = choice.upper()

if choice == "Y" :
    print("Continue")
elif choice == "N" :
    print("Stop")
else :
    print("Invalid choice")
```

Mẹo

Chuẩn hóa bằng `upper()` hoặc `lower()` để không phải kiểm tra cả chữ hoa và chữ thường.

Hai đường tròn giao nhau: ý tưởng kiểm tra

Để kiểm tra hai đường tròn có giao nhau, có thể so sánh khoảng cách giữa hai tâm với bán kính.

Ba đại lượng chính

- khoảng cách giữa hai tâm: `distance`;
- tổng bán kính: $r1 + r2$;
- độ chênh bán kính: $\text{abs}(r1 - r2)$.

Điều kiện giao nhau thông thường

$\text{abs}(r1 - r2) \leq \text{distance}$ and $\text{distance} \leq r1 + r2$

Ví dụ bổ sung: kiểm tra số lượng mua hàng

```
quantity = int(input("Quantity: "))

if quantity <= 0 :
    print("Quantity must be positive")
elif quantity > 100 :
    print("Quantity too large")
else :
    print("Accepted")
```

Mẫu tư duy

Kiểm tra lỗi trước, sau đó xử lý trường hợp hợp lệ.

Quiz 3.7: input validation

Một chương trình đọc số giờ làm việc trong tuần. Giá trị hợp lệ là từ 0 đến 80.

- ❶ Viết điều kiện phát hiện giá trị không hợp lệ.
- ❷ Nêu 5 ca kiểm thử nên dùng.
- ❸ Vì sao không nên tính lương trước khi kiểm tra số giờ hợp lệ?

Gợi ý đáp án Quiz 3.7

- 1 Điều kiện không hợp lệ: $\text{hours} < 0$ or $\text{hours} > 80$.
- 2 Các kiểm thử: -1, 0, 1, 80, 81.
- 3 Vì dữ liệu sai có thể tạo kết quả sai hoặc gây lỗi ở các bước xử lý sau.

Tổng kết: câu lệnh rẽ nhánh

- `if` chọn thực hiện một khối khi điều kiện đúng.
- `else` xử lý trường hợp điều kiện sai.
- `elif` phù hợp với nhiều phương án loại trừ lẫn nhau.
- Thụt lề xác định khối lệnh trong Python.
- Nên tránh lặp mã trong các nhánh.

Tổng kết: điều kiện và Boolean

- Toán tử quan hệ tạo ra giá trị True/False.
- `==` kiểm tra bằng nhau; `=` là phép gán.
- Không nên so sánh bằng tuyệt đối với số thực sau tính toán.
- `and`, `or`, `not` kết hợp điều kiện.
- Đánh giá ngắn mạch có thể tránh lỗi và giảm tính toán.

Tổng kết: kiểm thử và nhập liệu

- Mỗi nhánh quan trọng cần có ca kiểm thử.
- Giá trị biên thường là nơi dễ phát sinh lỗi.
- Kiểm tra dữ liệu đầu vào trước khi xử lý.
- Với văn bản, nên chuẩn hóa khi không cần phân biệt hoa/thường.
- Mô phỏng bằng tay giúp hiểu chương trình và tìm lỗi logic.

Quiz nhanh

- 1 Sự khác nhau giữa `=` và `==` là gì?
- 2 Khi nào nên dùng `elif` thay vì nhiều `if` độc lập?
- 3 Vì sao điều kiện `score < 0 and score > 100` luôn sai?
- 4 Giá trị biên của điều kiện `age >= 18` là những giá trị nào?
- 5 `not (A and B)` tương đương với biểu thức nào?

Bài tập trên lớp 1: phân loại điểm

Viết chương trình đọc một điểm số nguyên và in ra:

- Invalid nếu điểm ngoài khoảng 0–100;
- A nếu điểm từ 90 trở lên;
- B nếu điểm từ 80 đến dưới 90;
- C nếu điểm từ 70 đến dưới 80;
- D nếu điểm từ 60 đến dưới 70;
- F nếu điểm dưới 60.

Yêu cầu

Dùng `if/elif/else` và thiết kế ít nhất 6 ca kiểm thử.

Bài tập trên lớp 2: kiểm tra tên người dùng

Viết chương trình đọc username và kiểm tra:

- độ dài từ 6 đến 12 ký tự;
- không chứa khoảng trắng;
- chỉ gồm chữ cái và chữ số.

Nếu hợp lệ, in `Valid username`; ngược lại in thông báo lỗi phù hợp.

Gợi ý

Dùng `len()`, `" "` in username, `isalnum()`.

Quiz tổng hợp bổ sung

Trả lời nhanh.

- 1 Điều kiện nào kiểm tra x nằm ngoài đoạn 1–10?
- 2 Với `name = "An"`, vì sao `name == "an"` là `False`?
- 3 Khi nào nên chuẩn hóa chuỗi bằng `upper()` hoặc `lower()`?
- 4 Vì sao cần kiểm thử 49, 50, 51 với điều kiện `score >= 50`?
- 5 `not (x < 0 or x > 100)` tương đương với điều kiện nào?

Đáp án quiz tổng hợp bổ sung

- ❶ $x < 1 \text{ or } x > 10$.
- ❷ Vì Python phân biệt chữ hoa và chữ thường.
- ❸ Khi không muốn phân biệt chữ hoa/chữ thường trong dữ liệu nhập.
- ❹ Vì đó là các giá trị ngay dưới, đúng tại, và ngay trên ngưỡng.
- ❺ $x \geq 0 \text{ and } x \leq 100$.

Ví dụ tổng hợp: phân loại đơn hàng

```
member = input("Member? (Y/N): ").upper()
amount = float(input("Amount: "))

if amount < 0 :
    print("Invalid amount")
else :
    discount = 0
    if member == "Y" and amount >= 1000000 :
        discount = amount * 0.10
    elif member == "Y" :
        discount = amount * 0.05
    total = amount - discount
    print("Discount:", discount)
    print("Total:", total)
```

Hướng dẫn học tập

Sau bài này, khi gặp một bài toán có điều kiện, hãy luyện theo chu trình:

Hiểu bài toán → **Xác định nhánh** → **Viết điều kiện** → **Kiểm thử biên**

Thói quen quan trọng

Trước khi viết code, hãy tự hỏi: “Có những trường hợp nào khác nhau? Mỗi trường hợp cần kiểm thử bằng dữ liệu nào?”